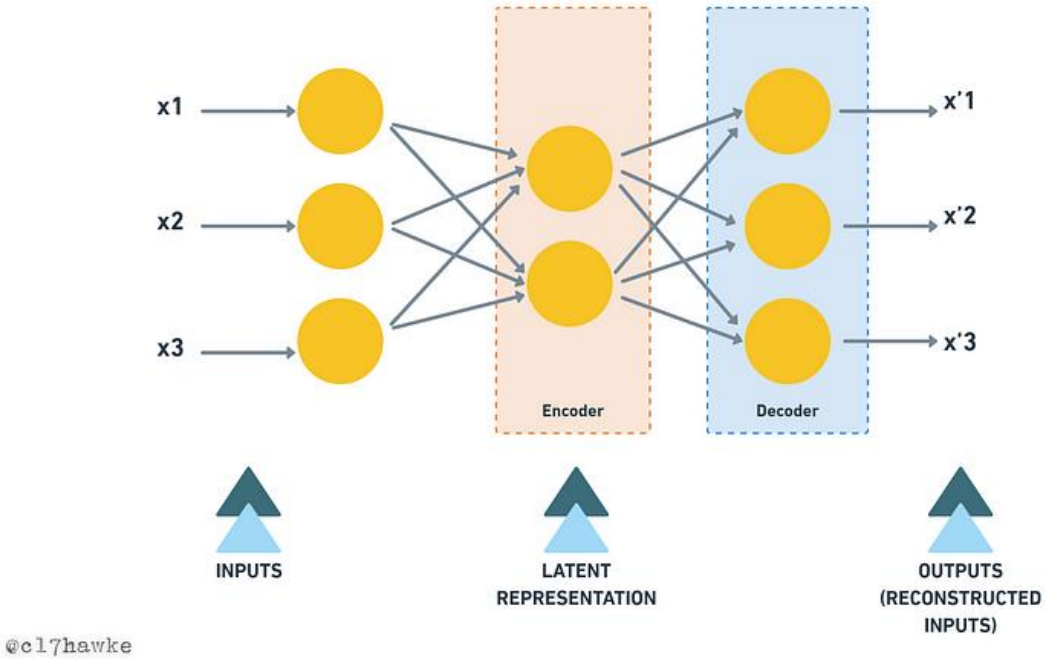
Introduction to Autoencoders



A SIMPLE AUTOENCODER

Autoencoders are a type of neural network that can be used for unsupervised learning tasks such as anomaly detection, image compression, and feature extraction. In this blog, we will delve into the basics of autoencoders and their Python implementation.

So, before we dive into autoencoders, let's first observe a few examples/analogies -

Example/analogies 01:

Consider the following sequences of numbers and observe which one of them is easier to remember —

Sequence 01: 6, 7, 5, 4, 9, 9, 2, 8…

Sequence 02: 2, 4, 6, 8, 10, 12, 14, 16…

I'm sure you have already observed immediately that Sequence 01 does not appear to follow any obvious pattern. It appears to be a random sequence of numbers.

And sequence 02 is easier to remember because it follows a clear pattern of adding 2 to the previous number in the sequence. Each number in the sequence is 2 more than the previous number. This is an arithmetic sequence with a common difference of 2 or it can be thought of as 2x where x implies the position of the number in sequence starting from x=1.

This makes Sequence 02 it easier to remember because there is a latent pattern involved which we identified immediately. It is worth noting that when a pattern is present within a sequence, it becomes easier for us to remember longer sequences.

In such cases, we only need to remember the pattern rather than the actual sequence, which makes the task simpler.

Example/analogies 02:

Imagine you are a nature enthusiast who wants to capture the beauty of a scenic waterfall on camera. You set up your camera, press record, and begin filming. The camera captures the waterfall's movement, the surrounding trees and rocks, and the changing light and shadows over time. All of this information is three-dimensional — there are height, width, and depth components to the scenery you're capturing.

Now, when you play back the video, you're watching a two-dimensional representation of that three-dimensional scene. The video's dimensions are reduced — you're only seeing a flat image on a screen. But even though you're missing one dimension of information, you're still able to comprehend the entire video and extract useful information from it. You can see the

movement of the waterfall and how it changes over time. You can observe the colors of the foliage and how they shift with light. You can even measure the height of the waterfall from the footage.

This is a classic example of dimensionality reduction. By converting a three-dimensional scene into a two-dimensional format (the video), we've reduced its complexity and made it easier to work with. This is a key advantage of dimensionality reduction techniques — they allow us to extract important information from complex data sets while streamlining the processing and analysis. So just like how we can still appreciate the beauty of a waterfall on a two-dimensional screen, we can still extract valuable insights from reduced-dimensional data sets.

Example/analogies 03:

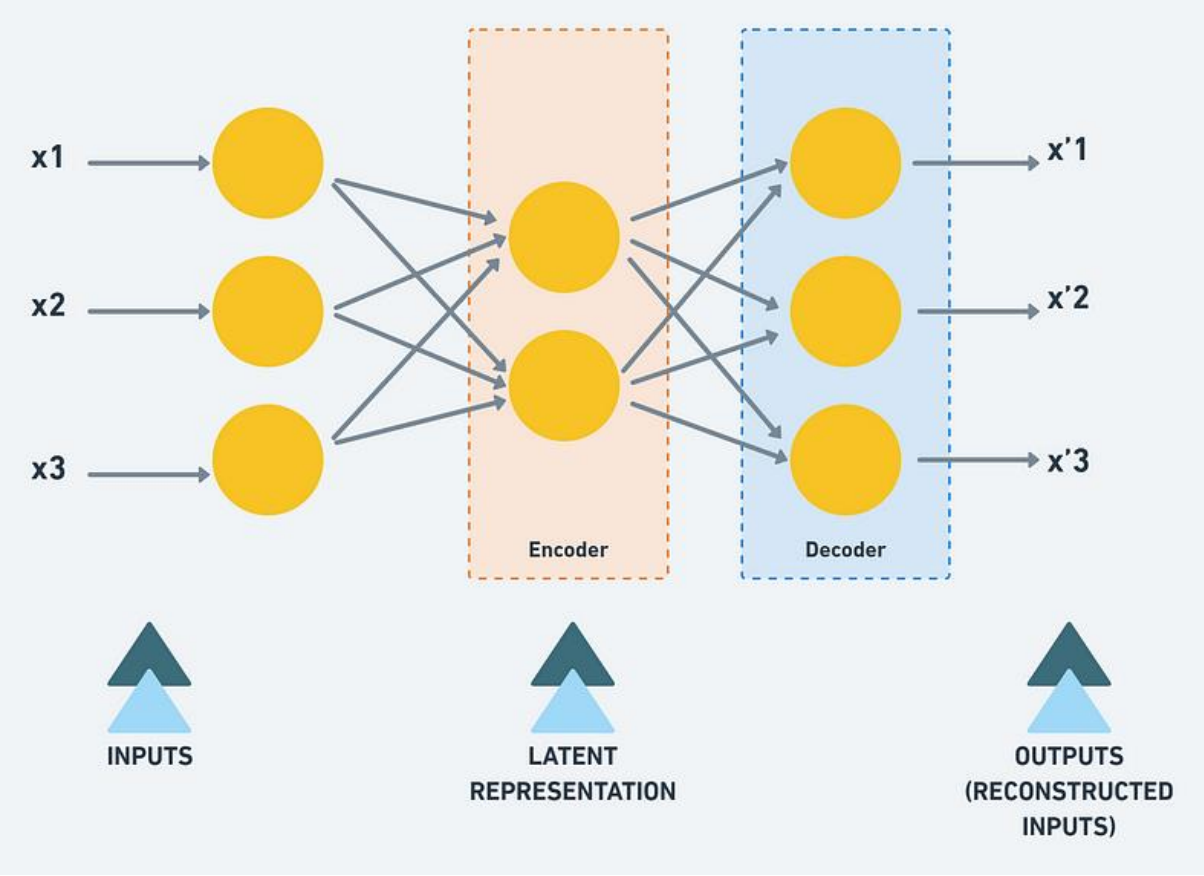
We all prefer lecture summaries or notes of chapters, instead of remembering every detail, we can extract the key ideas or concepts and summarize them into a more concise form. By doing so, we reduce the complexity of the information, making it easier to process, understand and apply. This approach allows us to retain the most important information and make informed decisions without being overwhelmed by unnecessary details. Which is analogous to dimensionality reduction techniques and analogous to these techniques we widely use in data science and machine learning to extract meaningful information from large datasets

In all these examples, we can observe that the aim is to reduce the amount of information needed to make informed decisions. This idea is similar to the concept of dimensionality reduction in machine learning, where techniques like Principal Component Analysis (PCA) are used to extract the most important features from complex data. Autoencoders also follow the same approach of condensing or reducing the dimensionality of the data. So now let's understand Autoencoders-

What are Autoencoders?

Autoencoders are a type of unsupervised learning technique used in machine learning. They are capable of finding hidden features or information in raw input data using artificial neural networks. By reducing the dimensionality of the data, autoencoders try to find latent or hidden representations. This hidden information is also known as latent or hidden features.

An autoencoder is a neural network that is trained to reconstruct its input. It consists of two parts: an encoder and a decoder. The encoder takes the input and compresses it into a lower-dimensional representation, while the decoder takes this compressed representation and reconstructs the original input. The goal of the autoencoder is to minimize the difference between the input and the reconstructed output. [Refer to the following figure —]

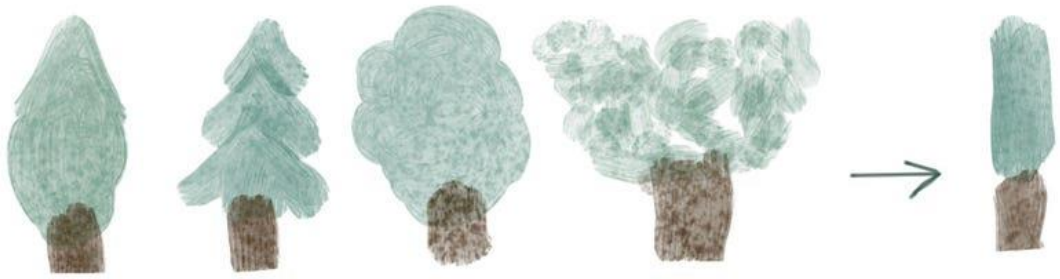


While reconstruction the encoder layer is forced to learn the important latent representation and this information can be used to reduce the dimension.

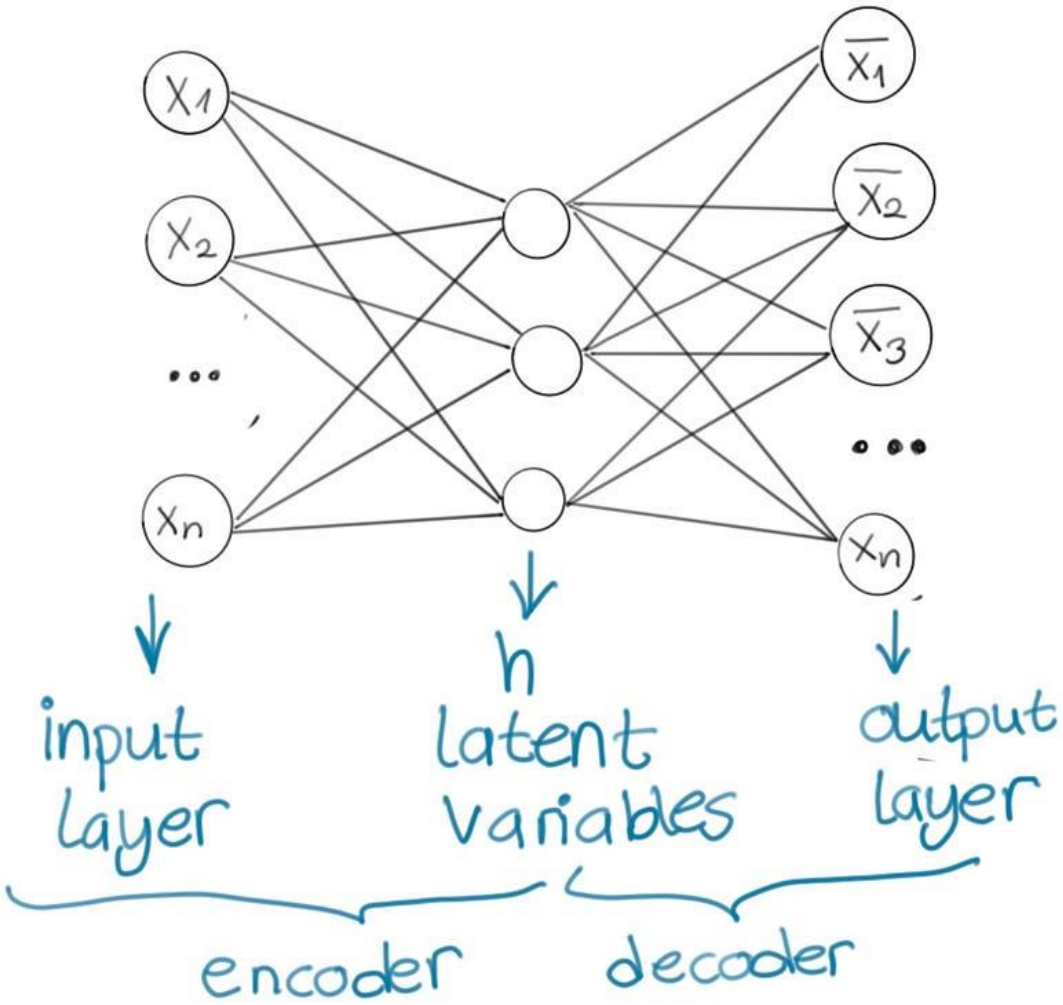
Autoencoder: Basic Ideas

Autoencoder is the type of a neural network that reconstructs an input from the output. The basic idea here is that we have our inputs, and we compress those inputs in such a manner that we have the most important features to reconstruct it back.

As humans, when we're asked to draw a tree with the least number of touches to the paper, (given that we've seen so many trees in our lifetime) we draw a line for the tree and couple of branches on top to provide an abstraction to how trees look like, this is what's being done with autoencoder.

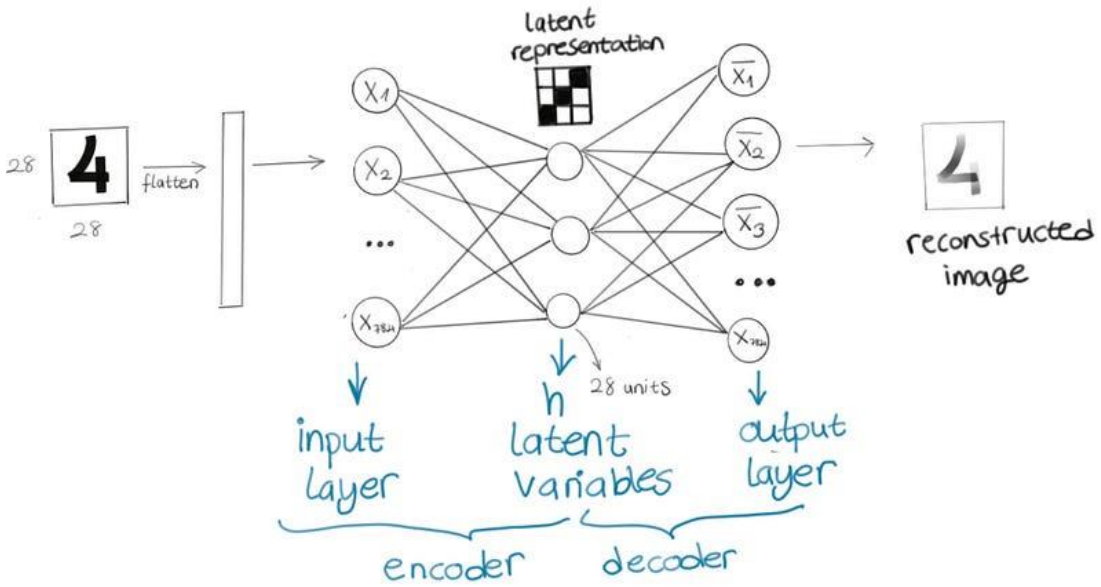


An average autoencoder looks like below:



Let’s take a solid case for image reconstruction.

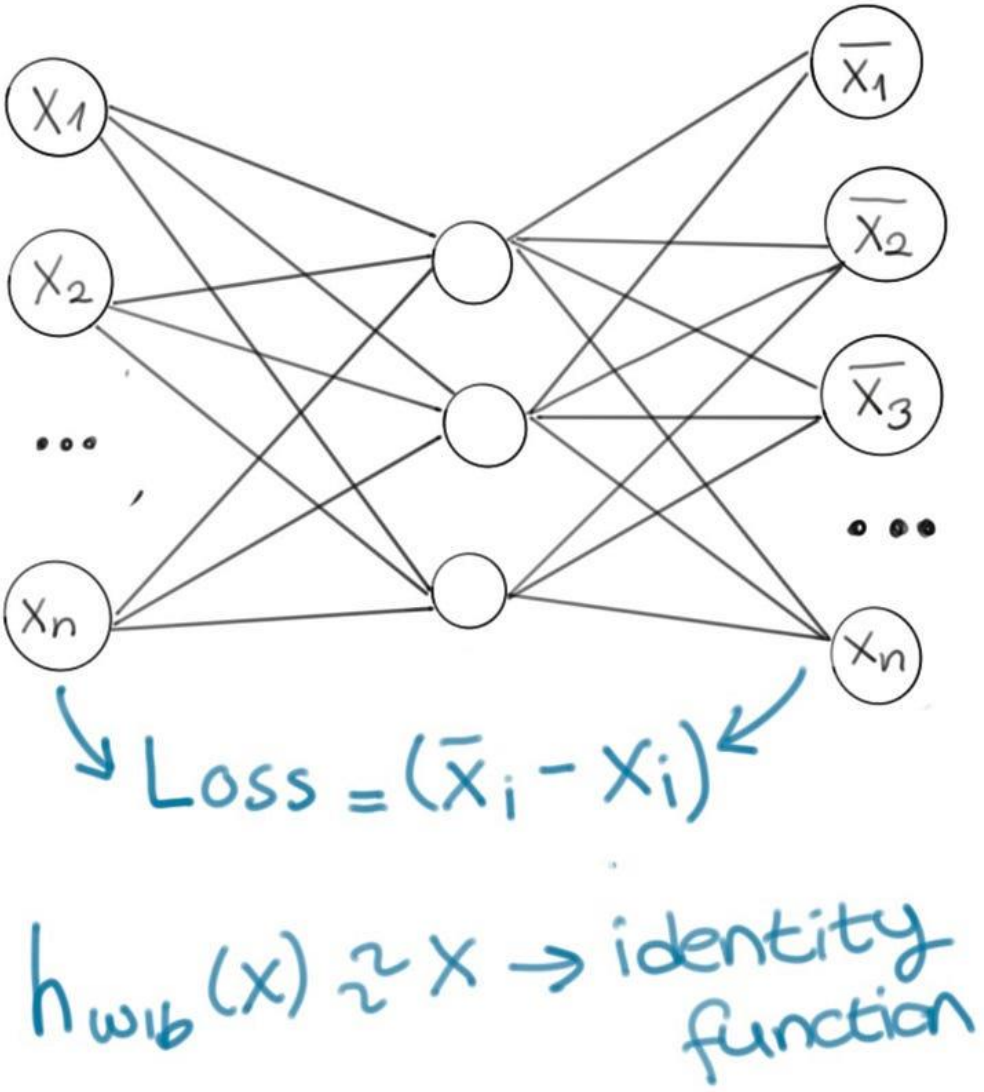
We have our input layer with 784 units (assuming we give 28x28 images) and we could simply stack a layer on top with 28 units, and our output layer will have 784 units again.



The first part is called “encoder” it encodes our inputs as latent variables, and second part is called “decoder” it will reconstruct our inputs from the latent variables.

The hidden layer having a smaller number of units will be enough to do the compression and get latent variables. This is called “undercomplete autoencoder” (we also have other types of autoencoders but this gives the main idea, we’ll go over them as well). So, in short, it’s just another feed forward neural network that has the following characteristics:

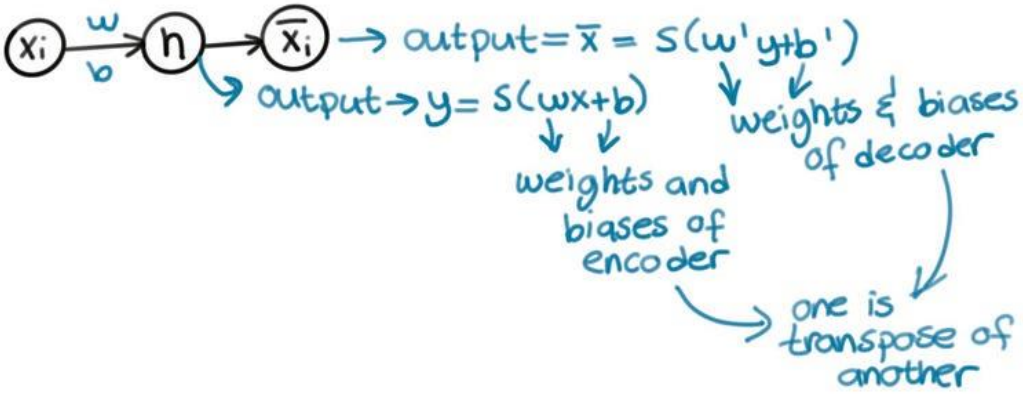
- input layer, hidden layer with a smaller number of units and output layer
- it’s unsupervised: we will pass our inputs, get the output and compare with input again
- Our loss function will be comparing the input to compressed and then reconstructed version of the input and see if the model is successful or not.



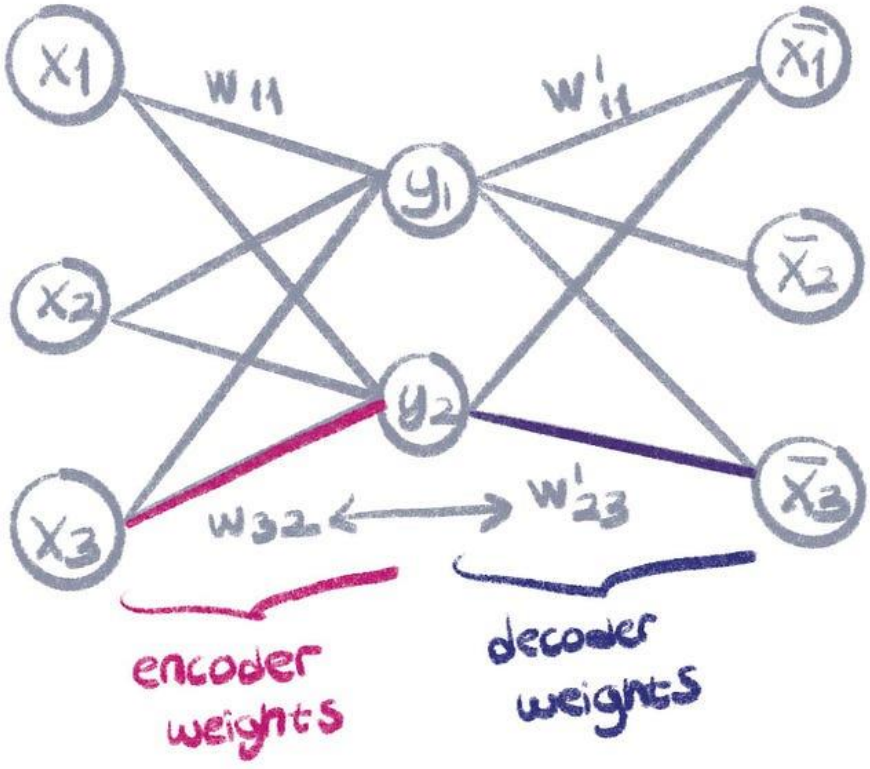
- An autoencoder that has a decoder with a linear layer essentially does the same thing as principal component analysis (even though training objective is to copy the input).

- Another core concept of autoencoder is weight tying. The weights of decoder are tied to weights of encoder. When you transpose the weight matrix of encoder, you get the weights of decoder. It's a common practice for decoder weights to be tied to encoder weights. This saves memory (using less parameters), and reduced

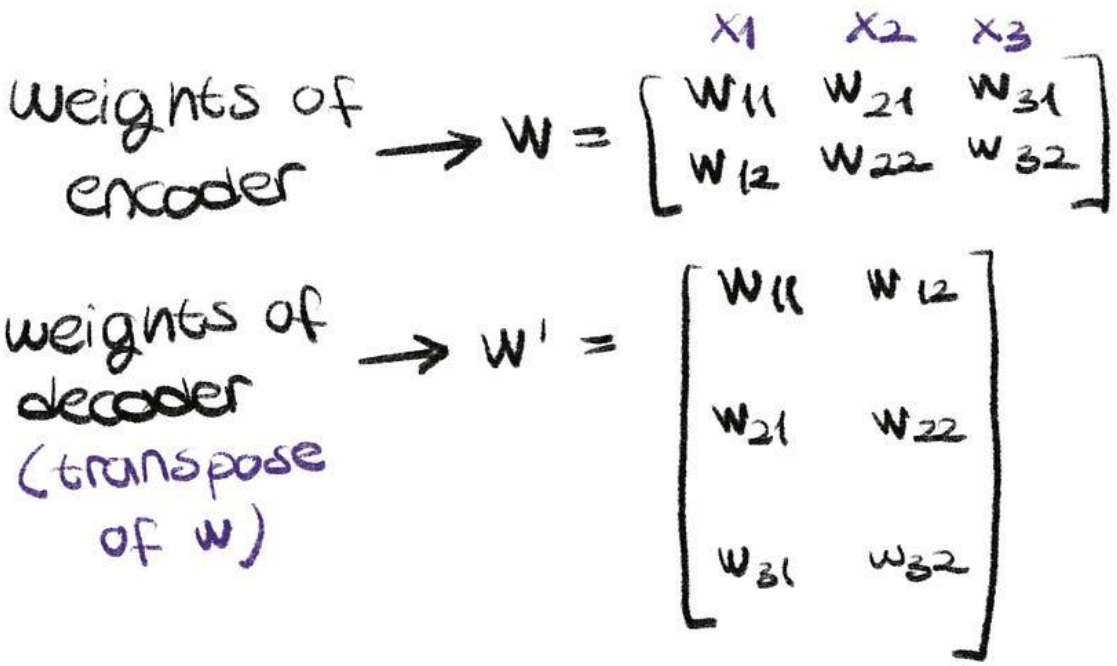
overfitting. I tried to explain my intuition below in multiple graphics. Let's take a look.



For the autoencoder below:



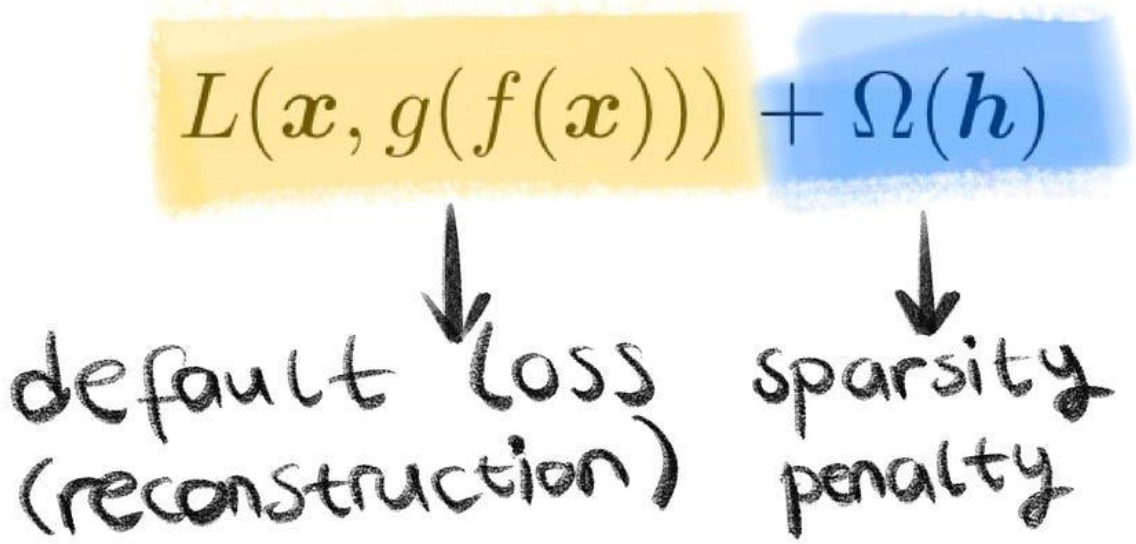
Weight matrix of encoder and decoder looks like this (you can skip this if you know what transpose means):



In contrast to undercomplete autoencoder, complete autoencoder has equal units, and overcomplete autoencoder has more units in latent dimension compared to encoder and decoder. This causes the model to not learn anything but rather overfit. In undercomplete autoencoders on the other hand, the encoder and decoder might be over capacitated with information if hidden layer is too small. To avoid overengineering this and add more functionalities to autoencoders, regularized autoencoders are introduced. These models have different loss functions that not only copy input to output, but make the model more robust to noisy, sparse or missing data. There are two types of regularized autoencoders, called denoising autoencoder and sparse autoencoder. We will not go through them in depth in this post since implementation doesn't differ a lot from the normal autoencoder.

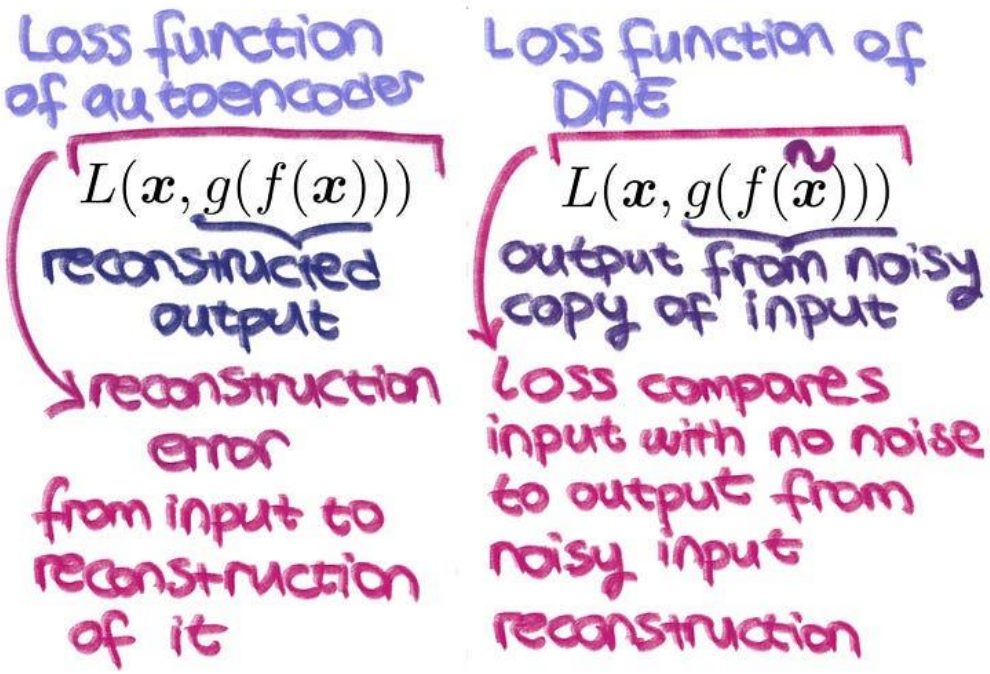
Sparse Autoencoder

Sparse autoencoders are autoencoders that have loss functions with a penalty for latent dimension (added to encoder output) on top of the reconstruction loss. These sparse features can be used to make the problem supervised, where the outputs depend on those features. This way, autoencoders can be used for problems like classification.



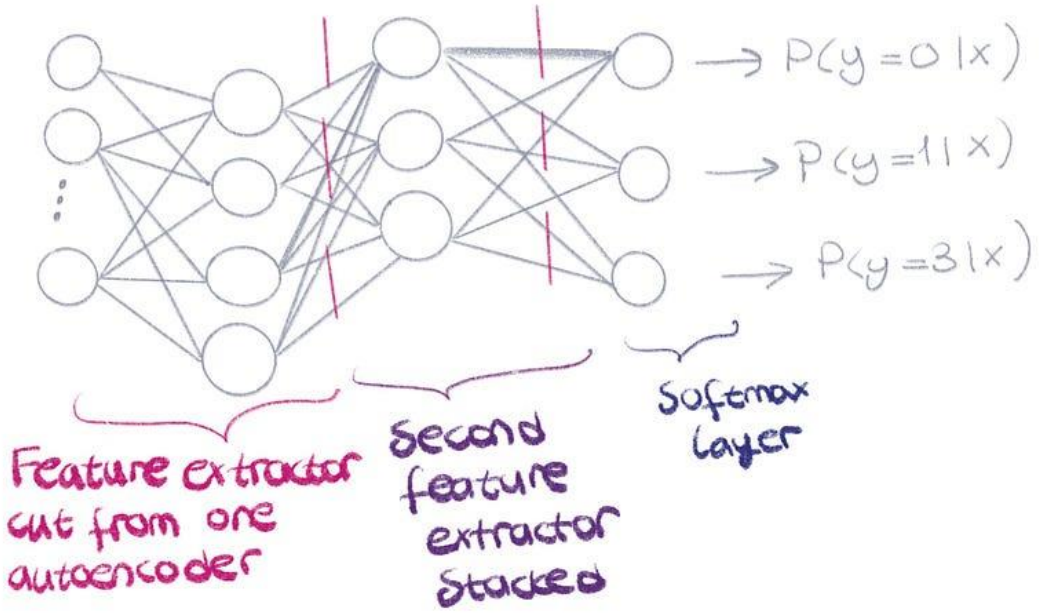
Denoising Autoencoder

Denoising autoencoders are type of autoencoders that remove the noise from a given input. To do this, we simply train the autoencoder with corrupted version of input with a noise and ask the model to output the original version of the input that doesn't have the noise. You can see the comparison of loss functions below. The implementation of this is same as normal autoencoder, except for the input.



Stacked Autoencoder

Before ReLU existed, vanishing gradients would make it impossible to train deep neural networks. For this, stacked autoencoders were created as a hacky workaround. One autoencoder was trained to learn the features of the training data, and then the decoder layer was cut and another encoder is added on top and the new network is trained. At the end, softmax layer was added to use these features for classification. This could be one of the early techniques to do transfer learning.



What are Autoencoders in Deep Learning

We all are well aware of the Supervised Machine Learning algorithms where ML algorithm tries to understand the relationship between input features and labels from the training data and is expected to automatically generate a similar kind of relationship between test data features and set of possible output labels. Although we solve lots of problems with supervised learning algorithms, not all problems are supervised in nature. There are problems where we just need the algorithm to learn representation (encoding) from the dataset without requiring labels. These algorithms are known as unsupervised learning ML algorithms. Autoencoders also come under unsupervised learning techniques.

Clustering algorithms (for example- KNN, KMeans, DBSCAN, etc.) are also examples of unsupervised learning techniques as they do not require labeled data to learn relationships/representations in data. Autoencoders are deep learning-based(Neural Networks) algorithms that are widely used to solve many complex tasks.

Here is a list of some of the popular tasks that are solved efficiently using autoencoder based deep learning models-

- Dimensionality Reduction
- Information Retrieval
- Anomaly Detection
- Image Processing
- Drug discovery
- Population synthesis
- Popularity prediction
- Machine Translation

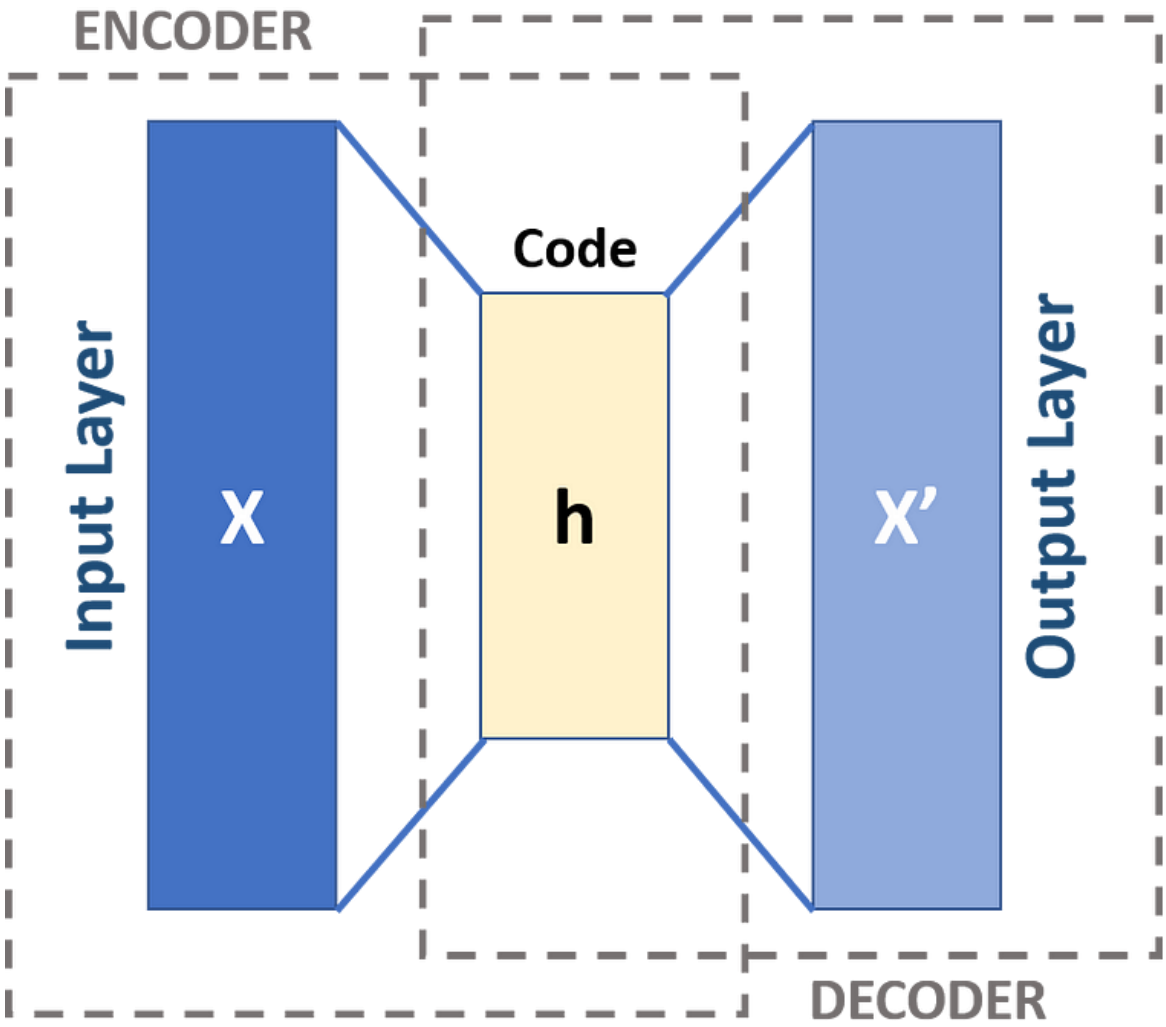
1. AutoEncoders in Keras and Deep Learning

I hope everyone is aware of sponge balls that are extensively used as stress balls. Stress balls (or hand exercise balls) are squeezed in hand and manipulated by fingers to relieve muscle tension and stress as prescribed in physical therapy.

Sponge balls are known for their elastic property which brings them into their original shape when squeezed forcefully. Due to this property, you can actually keep these balls in smaller boxes(in size) with force and they will acquire their original shape back (approximately) once the box is opened.

The idea behind autoencoders is very similar. An autoencoder will take a data point, squeeze to fit it in a smaller box(smaller data shape), and will be able to convert it back(to its original form) when required. As our data is not elastic like the stress balls, It will not get back to its original form perfectly which means that we will be losing some information in the process. An autoencoder is made up of two parts-Encoder and Decoder.

The Encoder part of an autoencoder is a neural network (non-recurrent generally) that takes the input data point and squeezes it into a lower dimension state (h: as shown in the below figure). This code(h) is expected to learn important information from the input data point. The Decoder part of the autoencoder is again a neural network that is supposed to take this hidden-code representation(h) of the data as input and reconstruct the original data point as output. Thus the inputs and outputs of an autoencoder are the same, and the most important thing is the hidden state(compressed representation of data) that learns important features from the data.



Autoencoders were primarily used for dimensionality reduction and feature learning in the past decades. But recently this technique is quite popular and is widely used in developing generative models (like-Generative Adversarial Networks (GANs)).

Dimensionality reduction is the task of representing high dimensional data points in lower dimensions without losing much information. In this manner, autoencoders are significantly related to the Principal Component Analysis(PCA) when used with linear activations. While the non-linearity(non-linear activations) makes the autoencoders much superior and capable of learning more complicated relationships in the data so that reconstruction is performed without losing any significant information.

3. Types of Autoencoders

There are various techniques(regularization techniques) to make autoencoders robust such that they learn the important features instead of just remembering and copying the input data into output.

Here are a few regularized variants of autoencoders that work really well in practice-

Sparse autoencoders (SAE)

Sparse autoencoders are typically used when we need to learn features for other tasks, such as classification. These autoencoders involve a sparsity penalty on the embedding layer(h-layer) in addition to the reconstruction error.

This penalty can be thought of as a regularization parameter to the simple autoencoder where the task was to simply copy the input into an output. This sparsity penalty makes the model learn better representative features for the dataset instead of just copying input into the output.

Denoising autoencoders (DAE)

Denoising autoencoders are trained by changing the reconstruction error term of the cost function(or loss). Traditionally the input data points and output data points are the same for an autoencoder but in the case of denoising autoencoders, the input data points are noisy in nature while the reconstructed data points are compared with the clean data point(without noise). In this manner, a DAE would learn to ignore(or remove) noise from the noisy dataset.

Denoising autoencoders have the ability to learn useful features from the input data and discard the useless information(like-noise) at the time of reconstruction. This makes them learn better representations of the data and they are trained to remove noise from noisy datasets.

Contractive autoencoders (CAE)

Denoising autoencoders typically work well when there is small and limited noise in the dataset, Contractive autoencoders on the other hand are robust to significant perturbations on the input data. Contractive autoencoders apply a contractive penalty on encoder output that encourages the derivatives of the encoder to be as small as possible. This penalty is basically the sum of squared elements of partial derivates from the encoder function.

Contractive autoencoders are trained to learn the distributions of input instead of copying it, thus they are robust to the slight variations in the inputs.

Variational autoencoders (VAE)

Variational autoencoders are different from the traditional autoencoders discussed above(like-denoising, sparse, contractive). These are generative models like Generative Adversarial Networks(GANs).

Traditional autoencoders basically learn latent representations of inputs using reconstruction error. The distribution of these latent representations is usually uneven(input-related). Variational autoencoders use reconstruction loss along with the KL-divergence loss. Here KL-divergence loss encourages the model to learn broader distributions of latent features.

These latent features from the Variational autoencoder are assumed to follow the Gaussian distribution when trained on sufficient training data. We can sample features from the latent distribution and form a generative model capable of generating new data without any input.